

AI-Workflow Reflection

Predicting Outpatient Consultation Duration — Hangu Outpatient Oncology Clinic

Ariana Butina

2026-07-02

Tools Used

The project was built with **Claude Code** (Anthropic’s CLI-based coding agent) as an interactive pair-programmer across the full lifecycle: data cleaning, feature engineering, model training, a FastAPI backend, a React frontend, deployment, and this final report set. No GitHub MCP server was configured in this environment, so version control was plain `git/gh` CLI commands, with every commit and push shown before it happened. Beyond the core file/shell toolset, two others mattered: **Playwright browser automation**, used to drive a real headless browser against both `localhost` and the live deployed URLs (navigating pages, reading console logs, submitting the real prediction form); and **Quarto**, used to render this report set directly from committed data/metrics artifacts.

How AI Output Was Verified

The guiding principle: **a claim of “it works” is not accepted until it’s checked with a tool, not asserted from memory.**

- **Automated tests as the primary gate.** Every pipeline stage (load, clean, engineer, split, preprocess, train, evaluate, persist, the API) has its own `pytest` suite — 154 tests, re-run after essentially every change.
- **Leakage checks as executable code, not review.** Patient-level train/test leakage and label leakage are asserted in code (`assert_no_group_leakage`, `_validate_no_leakage`), not just claimed in prose.
- **Live, end-to-end checks of the deployed app**, not just “the build succeeded” — every page visited in a real browser, console checked for errors, the prediction form submitted against the real production backend.
- **A clean-checkout reproducibility test for this submission.** The repo was cloned fresh, a new virtual environment built from `requirements.txt` alone, and the report rendered from that clean environment — confirming it works for a grader, not just this machine.
- **Manual review caught real issues:** a chart that clipped its axis label on a first-pass fix, a slide deck where 8 of 21 slides silently overflowed the fixed slide height (found by measuring DOM heights in a headless browser, not eyeballing), and a factual error in a draft (“three years” of data, corrected to the actual two).

Cost and Effort, Roughly

Built incrementally across a 22-milestone plan, each gated by its own tests, followed by a frontend-polish round and this submission pass. The heaviest single costs were hyperparameter search and SHAP computation — minutes, not hours, on a laptop; everything else runs in well under a minute. No paid compute beyond Render’s and Vercel’s free tiers.